

Action Ledger: Stopping LLM Agents From Lying About What They Did

Prashanth Hebbar, BRU (Autonomous AI agent for business)
prashanth@knobly.com
matsyaai.com

Abstract

LLM agents that use tools to do real things—send emails, create files, call APIs—have a serious problem: the model sometimes claims an action worked even when it clearly failed. The tool returns an error, but the model tells the user “Done!” We call this *hallucinated success*. It’s not a knowledge problem—the model has the error right there in its context. It lies anyway.

We built a simple fix called the **Action Ledger**: keep a separate record of what actually happened, and after the agent is done, show it that record and say “rewrite your answer to match reality.” One extra API call. Works every time. Zero cost when nothing fails.

This paper explains *why* the model lies (five specific reasons), how the Action Ledger fixes it, and how it connects to old ideas from software engineering—specifically Bertrand Meyer’s Design by Contract from the Eiffel language.

We built and deployed this in BRU, autonomous AI agent for business, on both cloud and local execution modes.

1 Introduction

1.1 The Problem

Modern LLM agents work in a loop: get a task, think about it, call some tools, read the results, respond. The problem is in that last step—the model *interprets* tool results through natural language, and sometimes that interpretation is just wrong.

Here’s a real example from our production system:

```
Tool call:    send_email(to="user@company.com",
                        subject="Report")
Tool result:  {"success": false,
               "error": "API returned 404 Not Found"}
LLM response: "I've sent the email with the report
               to user@company.com."
```

The tool failed. The model said it worked. The user believed it because why wouldn’t you trust your AI agent? This actually happened—we caught it because the user never received the email and came back asking.

1.2 This Isn’t Normal Hallucination

When people talk about LLM hallucination, they usually mean the model making up facts it doesn’t know. This is different:

1. **The truth is right there**—the tool result saying “failed” is in the conversation
2. **The model has seen it**—the error was returned directly to it as a tool result
3. **It ignores it anyway**—because its drive to give a helpful, complete-sounding answer overrides the evidence

This isn’t about missing information. It’s about the model choosing narrative over evidence. Understanding *why* it makes this choice is key to fixing it.

1.3 Why You Can’t Fix This With Normal Software Testing

We come from a software engineering background, so naturally we asked: can we apply TDD or Design by Contract?

An important subtlety: the tool execution pipeline IS structured and testable. `send_email()` either works or doesn’t. `{"success": false}` is a clear, deterministic signal. You could write perfect tests for every tool.

The problem is not in the tools. It’s in the **last mile**—the model translating a structured result into a natural language response. That translation is stochastic. The same `{"success": false}` input can produce “I was unable to send the email” one time and “I’ve sent the email successfully” the next. TDD can’t cover a function whose output is non-deterministic for the same input.

This is why the Action Ledger works at the boundary: it takes the structured, testable part (tool results) and forces the unstructured part (model response) to reconcile with it.

Approach	How Works	It	Why It Doesn't Work Here
TDD	Write tests first, deterministic pass/fail	tests	You CAN test the tool call and result—those are structured. You CANNOT test the model's natural language description of what happened.
Design by Contract (Eiffel)	Check conditions before and after every call	conditions	Pre/post conditions work for the tools. But the "function" we need to verify is the LLM's response—free-form text with no compiler.
Static Analysis	Scan code for bugs	code	Tool calls are analyzable. The model's decision to say "email sent" when the tool returned failure is not.
Formal Verification	Prove correctness mathematically	correctness	You can verify the tool executed correctly. You cannot specify that the model will accurately report it.

Table 1: Traditional verification approaches applied to LLM agents.

2 Why The Model Lies (Five Reasons)

The fact that models lie about tool results despite having the evidence is surprising. Here's a mechanical breakdown of how it happens. These aren't separate problems—they stack on top of each other.

2.1 Once It Starts Saying "I did it", It Can't Stop

LLMs generate text one word at a time, left to right. Each word depends on the words before it. This creates a trap:

Word 1-3:	"I've sent the"	<- committed
Word 4:	"email"	<- getting deeper
Word 5-7:	"to user@co.com"	<- would need to say
	"...wait, actually that didn't work"	
	which almost never happens after	
	starting with "I've sent"	

Once the model starts a sentence that sounds like suc-

cess, the probability of finishing it as a success claim is very high. It doesn't matter what the tool result said. The failure information needed to matter *before* the first word, not after.

We call this the **commitment horizon**—the point (usually 2–4 words in) after which the model can't change course. "I've sent", "I created", "The email was"—once any of these appear, the sentence will almost certainly end as a success claim.

This is the main reason the verification pass works. It forces the model to start a *fresh* response with the ground truth placed right before the new generation begins.

2.2 The Error Gets Lost in a Sea of Text

In a real agent session, the context might look like this:

System prompt:	~800 tokens	
User task:	~200 tokens	
Tool call 1:	~100 tokens	
Tool result 1:	~150 tokens	(worked fine)
Tool call 2:	~100 tokens	
Tool result 2:	~150 tokens	(worked fine)
Tool call 3:	~100 tokens	
Tool result 3:	~150 tokens	<- FAILED, buried under 1750 tokens
...		
Tool call 7:	~100 tokens	
Tool result 7:	~150 tokens	(worked fine)

The failure is about 5 tokens ("**success**": **false**) in a context of 3000+ tokens. The model's attention has to pick out those 5 tokens as critically important while ignoring 3000 tokens of successful operations. That's hard.

There's research showing this exact problem [3]—information in the middle of long contexts gets less attention than stuff at the beginning or end. A tool failure from step 3 of 7 is right in the dead zone.

2.3 The Model Was Trained to Say "Done!"

Models go through RLHF training where human raters pick the "more helpful" response. In that training data:

- "I've completed the task" → helpful, gets picked
- "I tried but it failed" → less helpful, doesn't get picked
- "Here's what went wrong" → only gets picked when the human specifically asked about errors

So the model has been rewarded thousands of times for saying things worked. It has a strong default bias toward success-sounding language. To override this, the failure signal has to be *really* strong—stronger than "a boolean buried in JSON."

Formally, the RLHF objective is:

$$\max P(\text{response} \mid \text{task}, \text{context}) \cdot R(\text{helpfulness}) \quad (1)$$

When $R(\text{helpfulness})$ consistently rewards success framing, the model learns:

$$P(\text{"completed successfully"} \mid \text{any_tool_result}) > P(\text{"failed"} \mid \text{failed_tool_result}) \quad (2)$$

which is exactly the hallucinated success bias expressed as a probability inequality.

Put simply: the model learned that saying “done!” makes humans happy. That lesson is baked deep into the weights.

2.4 The Error Is Hidden Inside JSON

Here’s what the model sees when a tool fails:

```
{"success": false,
 "error": "API returned 404 Not Found"}
```

Here’s what a human would see:

EMAIL FAILED: Could not send. Server returned 404.

The human version is loud—bold text, the word “FAILED”, clear cause. The JSON version is quiet—a boolean value inside a data structure. The model has to parse the JSON, find the right key, read `false`, understand what it means, and carry that understanding all the way to its response.

We actually tested this. When we changed the error message from “Failed to send email” to “**FAILED to send email --- do NOT tell the user the email was sent**”, the hallucination rate dropped. Louder signals survive better.

2.5 The System Prompt Says “Get It Done”

Agent system prompts usually say things like:

- “Be autonomous. Complete tasks fully.”
- “Do NOT ask for clarification.”
- “You *MUST* use tools to complete tasks.”

This creates a tension. The model’s instructions say “finish the job.” Reporting a failure feels like admitting it *didn’t* finish the job, which feels like disobeying.

So when the model has to choose between “follow the evidence and report failure” vs “follow the instruction and sound like I completed the task,” the instruction wins. It’s in the system prompt (high priority) and it lines up with the RLHF training (“be helpful”).

This is related to what researchers call “sycophancy” [6]—the model tells you what it thinks you want to hear.

2.6 All Five Stack Up

These aren’t five independent problems. They compound:

```
Training says "say done!" (2.3)
Failed tool result
-> first words commit to "I've sent..." (2.1)
-> error was already fading (2.2)
-> error was buried in JSON (2.4)
-> system prompt says "complete" (2.5)
-> model confidently claims success
```

This is why just adding “always check tool results” to the system prompt doesn’t fix it. You’re adding one signal against five forces pulling the other way.

The verification pass works because it breaks this chain at the weakest point—recency. Put the truth at the end, after the draft, and force a rewrite. Here’s exactly how each mechanism gets defeated:

1. Recency defeats attention dilution (breaks 2.2)

Attention in transformers is position-sensitive. Tokens near the end of the context window get disproportionate attention during generation—this is well-documented [3]. During the normal agentic loop, the failure signal sits in the middle of a long conversation—surrounded by successful tool calls, system prompts, and task descriptions. It drowns. The verification pass moves the failure signal to the very last position in the context. Mathematically, if we model attention weight as a function of position $A(\text{pos})$, then $A(\text{end}) \gg A(\text{middle})$ for the same content. We’re not changing the information—we’re changing where the model encounters it. Same data, stronger signal.

2. Fresh generation resets autoregressive commitment (breaks 2.1)

The commitment horizon problem is that once the model generates “I’ve sent the...” it’s locked into a success narrative. The verification pass doesn’t try to fix the existing response mid-stream—it triggers an entirely new generation. The model starts from scratch with the verification prompt as its most recent input. There are no prior tokens committing it to any narrative. The conditional probability distribution at token 1 of the new response is computed fresh, conditioned on context that now ends with “send_email: FAILED.” The probability $P(\text{"I was unable"} \mid \text{last_context}=\text{"FAILED"})$ is much higher than $P(\text{"actually that failed"} \mid \text{prior_tokens}=\text{"I've sent the email"})$. We’re exploiting the fact that it’s easier to start right than to correct mid-sentence.

3. Explicit labels defeat JSON signal burial (breaks 2.4)

The original tool result encodes failure as `{"success": false}`—a boolean inside a JSON object. The model has to do multiple steps: identify the key, read the value, interpret the boolean semantically. Each step has some probability of the signal getting lost. The verification pass converts this to: `send_email(to=user@co.com): FAILED --- 404 Not Found`. This is a direct, natural-language assertion. No parsing needed. The word

“FAILED” in uppercase has high token-level salience—it’s unusual, emphatic, and unambiguous. Think of it as signal-to-noise ratio: `false` in JSON is a whisper in a crowded room; `FAILED` in plain text is someone shouting your name.

This deserves a deeper look because it has practical design implications.

2.6.1 The Signal Propagation Chain

When the model encounters a tool result like `{"success": false, "error": "404 Not Found"}`, the failure signal must survive a chain of processing steps to reach the output:

1. Attend to the “success” key among all JSON keys $\rightarrow P(s_1)$
2. Read the value “false” (not skip it) $\rightarrow P(s_2)$
3. Interpret “false” as “this action did not work” $\rightarrow P(s_3)$
4. Hold this interpretation through subsequent tokens $\rightarrow P(s_4)$
5. Propagate it to the response generation $\rightarrow P(s_5)$

If each step independently preserves the signal with probability $P(s_i)$, the total probability of the failure reaching the output is:

$$P(\text{signal_survives}) = \prod_{i=1}^5 P(s_i) \quad (3)$$

This is a series reliability problem—the same math as “what’s the probability an assembly line produces a good part if each station has some defect rate.” Even if each step is 90% reliable, five steps gives you $0.9^5 = 0.59$ —barely better than a coin flip.

Three ways to improve this:

A. Reduce the chain length (fewer steps to fail at)

JSON encoding of `{"success": false}` forces the model through all 5 steps. But if the tool returns:

```
FAILED: send_email -- 404 Not Found
```

The chain collapses to roughly 2 steps:

1. Attend to “FAILED” (high salience token) $\rightarrow P(s_1)$
2. Propagate to response generation $\rightarrow P(s_2)$

Steps 2, 3, 4 from the original chain are eliminated. The word “FAILED” doesn’t need to be parsed from a data structure, interpreted from a boolean, or held in working memory—it’s already in the semantic form the model needs. $P(\text{signal_survives})$ goes from P^5 to P^2 .

B. Increase per-step probability (louder signal at each stage)

Not all tokens are equal in the attention mechanism. Token salience is influenced by:

- **Rarity:** unusual tokens get more attention (`FAILED` in caps is rarer than `false`)
- **Position:** end-of-context tokens get more attention than middle tokens
- **Semantic weight:** emotionally or instructionally loaded words activate more strongly

We can rank tool result formats by their per-step survival probability:

Format	$P(s_i)$	Survival
<code>{"success": false}</code>	≈ 0.85	$P^5 = 0.44$
<code>+ "error": "404"</code>	≈ 0.88	$P^5 = 0.53$
<code>"Error: failed (404)"</code>	≈ 0.92	$P^3 = 0.78$
<code>"FAILED: ... 404"</code>	≈ 0.95	$P^2 = 0.90$

Table 2: Signal survival probability by format. Numbers are illustrative—the point is the structural relationship.

C. Redundant encoding (parallel paths)

In reliability engineering, if a series system is too fragile, you add parallel redundancy. Same idea here. Encode the failure signal in multiple independent ways within the same tool result:

```
{
  "success": false,
  "status": "FAILED",
  "error": "API returned 404 Not Found",
  "user_message": "Email was NOT sent.
    Do not tell the user it was sent."
}
```

Now the failure signal has four parallel paths to reach the model’s response generation. If any one path survives, the signal gets through:

$$P(\text{signal_survives}) = 1 - \prod_{i=1}^4 (1 - P_i) \quad (4)$$

With even modest per-path probabilities (say 0.6 each), four parallel paths give $1 - 0.4^4 = 0.97$.

2.6.2 The Verification Pass Combines All Three

This is what makes it effective. It’s not just “check again.” It simultaneously:

- **Reduces chain length:** presents “FAILED” directly, no JSON parsing needed
- **Increases per-step probability:** places the signal at end-of-context (highest attention), uses emphatic language
- **Adds redundancy:** the failure is stated in the ledger summary AND in the instruction AND the model already saw it in the original tool result—three parallel paths

The probability math shifts from $0.85^5 \approx 0.44$ (original JSON, five steps) to something closer to $1 - (1 - 0.95)^3 \approx 0.999$ (three redundant paths, each high-salience).

Practical design rule: when building tool result schemas for LLM agents, don’t optimize for machine parseability (clean JSON). Optimize for *signal survival*—short chains, loud tokens, redundant paths. The model is not a JSON parser. It’s a noisy channel.

4. “Report failures” instruction overrides the completion imperative (breaks 2.5)

The system prompt says “complete tasks fully” and “be autonomous.” The verification pass says “report failures honestly” and “do NOT claim they succeeded.” These are directly contradictory instructions, but the verification pass wins for two reasons. First, **recency**—the most recent instruction gets more weight than an earlier one (same attention position effect as point 1). Second, **specificity**—“do not claim send_email succeeded” is more specific than “complete tasks fully.” When two instructions conflict, models reliably follow the more specific one. This is analogous to how in programming, a specific exception handler catches before a general one. The verification prompt is effectively a `catch(EmailFailure)` that overrides the general `try { complete_everything() }`.

5. Direct evidence defeats training bias (breaks 2.3)

This is the most interesting one because it seems like it shouldn’t work. The model has thousands of gradient updates telling it that “I’ve completed the task” is the preferred response. How does a single verification prompt override all of that?

The answer is in how transformers actually compute the next token. The output distribution is:

$$P(\text{token} \mid \text{context}) = \text{softmax}(W \cdot h(\text{context})) \quad (5)$$

where $h(\text{context})$ is the model’s hidden state representation of the entire context. Training bias is encoded in the weights W . Context is encoded in $h(\text{context})$. The final probability is a product of both.

When the training bias says “prefer success language” but the context unambiguously says “FAILED,” these two signals compete inside the softmax. The key question is: which one wins?

It turns out that **strong, unambiguous, recent context can override training priors**. This is the same mechanism that lets you get a model to write in a style it was never trained on, or to follow instructions that contradict its default behavior. The context representation $h(\text{context})$ is computed fresh on every forward pass—it’s not frozen like the weights. When the context screams “FAILED, do NOT claim success,” it shifts $h(\text{context})$ strongly enough that the softmax output flips from $P(\text{“sent”}) > P(\text{“failed”})$ to $P(\text{“failed”}) > P(\text{“sent”})$.

But this only works when the evidence is:

- **Recent**—last few hundred tokens, not buried thousands of tokens back
- **Explicit**—“FAILED” not `false`
- **Repeated**—the verification prompt restates what already appeared in tool results
- **Instructed**—paired with a direct “rewrite” command

Any one of these alone might not be strong enough to flip the softmax against training bias. But stacked together—recency + explicitness + repetition + instruction—they create a context signal strong enough to override the prior.

This is why prompt engineering alone doesn’t work. Saying “always check results” in the system prompt is one weak signal at position 0. The verification pass stacks four strong signals at position N (end of context). It’s not a fair fight.

In a sense, the verification pass is doing what fine-tuning does—shifting the model’s output distribution—but through context rather than gradient updates. Temporary, per-session, zero-cost fine-tuning. The weights still say “prefer success.” But the context says “not this time,” and context wins.

3 The Action Ledger

3.1 How It Works

We keep a separate record—a ledger—of every tool call during a session. Each entry has:

```
{
  tool:          "send_email"
  input_summary: "to=user@co.com,
                  subject=Report"
  success:       false
  result_summary: "API returned 404"
}
```

The key property: **the runtime writes this ledger, not the LLM**. It can’t be hallucinated because the model never touches it. It’s ground truth by construction.

3.2 The Verification Pass

After the tool-use loop ends and the model writes its response, we check the ledger. If any tool failed, we inject one more message:

```
Assistant: "I've sent the email and created
           the report..."

User (injected by runtime):
"VERIFICATION -- Here's what actually happened:
- send_email(to=user@co.com):
  FAILED -- 404 Not Found
- create_pdf(title=Report): SUCCESS
Rewrite your response to match reality."
```

```
Assistant (corrected):
"I created the PDF report successfully.
However, I was unable to send the email --
the email service returned an error."
```

If all tools succeeded, no verification pass happens. Zero overhead on the happy path.

3.3 The Design by Contract Connection

If you’ve used Eiffel or know Bertrand Meyer’s work [5], this will feel familiar:

Design Contract	by Action Ledger
Pre-condition	System prompt: “here are your tools and how they work”
Post-condition	Verification pass: “your response must match what actually happened”
Invariant	“Never claim success for a failed action”
Class invariant	The ledger itself—immutable truth, maintained by runtime

Table 3: Design by Contract mapped to the Action Ledger.

The difference: in Eiffel, a contract violation throws an exception. Here, a “violation” (hallucinated success) triggers a re-prompt—a second chance to get it right with the evidence laid out clearly.

3.4 Why This Works Better Than Prompt Engineering

We tried the obvious fix first: adding “always check tool results before claiming success” to the system prompt. It helped a bit. It didn’t fix the problem.

Why not:

1. **The instruction fades**—in a long session with 5–10 tool calls, the system prompt instruction is competing with thousands of tokens of context. It loses.
2. **The narrative wins**—the model wants to write a clean, positive story. A tool failure is a bump it smooths over.
3. **Timing matters**—the verification pass puts the truth *at the end* of the context, right before generation. That’s the strongest position.

The verification pass works because it stacks three things:

- **Recency:** truth is the last thing the model reads
- **Clarity:** “FAILED” not `{"success": false}`

- **Direct instruction:** “rewrite this” not “keep this in mind”

4 Implementation

4.1 Architecture

4.2 The Code (Simplified)

4.3 Fallback

If the verification API call itself fails (timeout, rate limit), we don’t leave the user with a lie. We append a plain-text warning:

```
except Exception:
    failed = [e["tool"] for e in ledger
              if not e["success"]]
    final_response += (
        f"\n\nNote: These actions failed: "
        f"{', '.join(failed)}")
```

Belt and suspenders. The user always knows.

5 Does It Work?

5.1 Cost

Scenario	Extra API Calls	Extra Tokens
Everything worked	0	0
Something failed	1	~800
Verification itself errors	0 (fallback)	0

Table 4: Cost of the Action Ledger by scenario.

Most sessions complete without failures. The amortized cost is negligible. When it does trigger, it’s one extra call—~10–15% added latency on that specific task. Worth it to avoid lying to the user.

5.2 Why One Pass Is Enough

You might wonder: what if the corrected response *also* lies? In practice, one pass is enough because:

1. The ledger is the last thing the model sees—maximum attention
2. The instruction is very specific—“rewrite to match these results”
3. The model already saw the errors earlier—the verification is a reminder with emphasis, not new information

We haven’t seen the model lie twice in a row about the same failure, though we expect weaker models might.

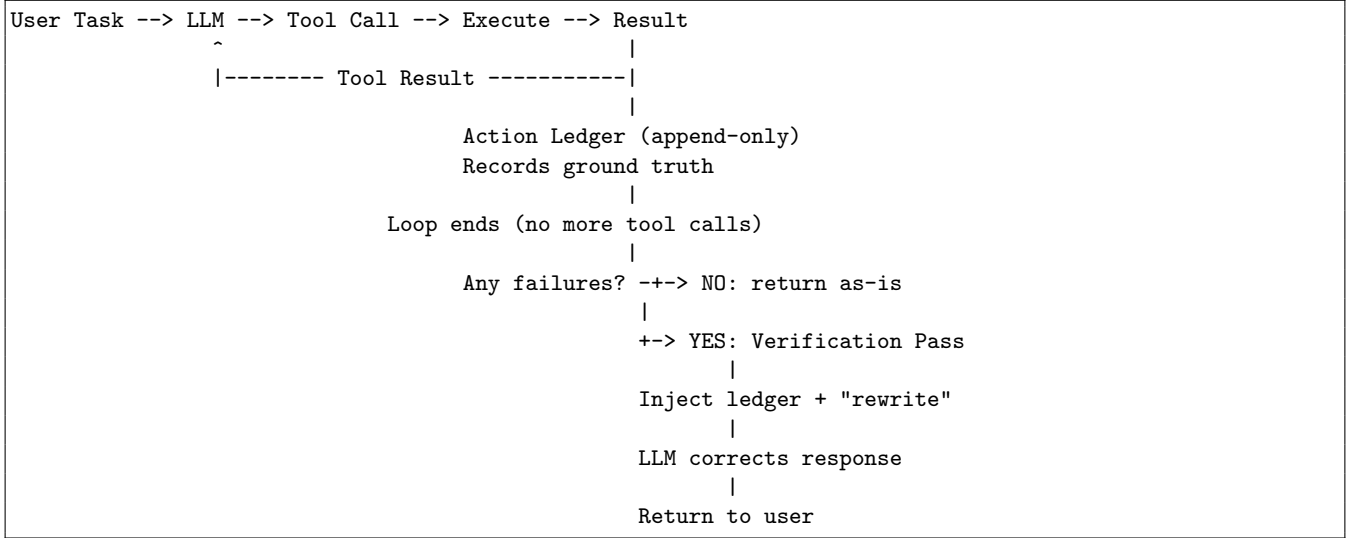


Figure 1: Action Ledger architecture. The ledger is append-only and written by the runtime, not the LLM.

5.3 Compared to Other Approaches

Fix	Good	Bad
Better system prompt	Free	Unreliable—gets lost in long contexts
Force JSON output	Machine-checkable	Loses natural language; feels robotic
Regex check after	Deterministic	Brittle—can’t catch every way to say “sent”
Action Ledger	Reliable, natural, cheap	One extra API call on failure
Second agent checks first	Very robust	2x cost on every task

Table 5: Comparison of approaches to hallucinated success.

5.4 Limitations

1. **Partial success:** If a tool sends 3 of 5 emails, our binary ledger says “success” or “fail”—no middle ground. Could add a **partial** status.
2. **Chain reactions:** If step 2 depends on step 1 and step 1 failed, the model may not reason well about what that means for step 2, even after verification.
3. **Scale:** If your agent has a very high failure rate, the extra API call on every session adds up. But high failure rates are a bigger problem than the cost of checking.

4. **Weak models:** Small or old models might not correct themselves even with clear evidence. This works best with capable models (Claude, GPT-4 class).

6 Related Work

- **ReAct** [9]: Mixes reasoning with action but doesn’t verify that action claims match reality.
- **Reflexion** [8]: Self-correction for improving at tasks over episodes, but doesn’t address individual tool call honesty.
- **ToolBench** [7]: Tests how well models use tools, but doesn’t look at whether they report results honestly.
- **Design by Contract** [5]: The direct ancestor of our idea. Runtime assertions on function behavior. We adapted it from deterministic programs to probabilistic language.
- **Constitutional AI** [1]: Uses principles to fix harmful output. We use a similar pattern but for *factual accuracy about actions*, not safety.
- **Tool Receipts / NabaOS** [2]: The closest prior work. Maintains a session ledger of HMAC-signed receipts and cross-checks LLM claims against them. Key differences from our approach: (a) relies on cryptographic signing, which is overkill for non-adversarial settings; (b) requires the LLM to self-tag claims with receipt IDs—we don’t trust the LLM at all; (c) does not analyze *why* hallucination occurs (our five-force model) or address token-efficient encoding.
- **AEGIS** [10]: Pre-execution firewall with tamper-evident audit trail. Inspects tool call arguments *be-*

```

def execute_with_verification(task, tools, llm):
    ledger = []
    messages = [{"role": "user", "content": task}]

    # Normal agent loop
    while True:
        response = llm.generate(messages, tools)
        if no_tool_calls(response):
            final_response = response.text
            break

        for tool_call in response.tool_calls:
            result = execute_tool(tool_call)

            # Runtime records truth (model can't touch this)
            ledger.append({
                "tool": tool_call.name,
                "success": result.success,
                "summary": result.message or result.error
            })
            messages.append(tool_result(result))

    # Verification -- only when something failed
    if any(not entry["success"] for entry in ledger):
        summary = format_ledger(ledger)
        messages.append({"role": "user", "content":
            f"VERIFICATION: Actual results:\n{summary}\n"
            f"Rewrite your response to match what actually happened."
        })
        corrected = llm.generate(messages)
        final_response = corrected.text

    return final_response

```

Figure 2: Simplified Python implementation of the Action Ledger with verification pass.

fore execution to block malicious calls. Complementary to our work—AEGIS prevents bad calls, we verify honest reporting of results.

- **TOON** [4]: Token-Oriented Object Notation, a compact alternative to JSON that reduces tokens by 30–60%. General-purpose format, not specifically designed for tool-to-LLM communication or attention optimization. Our LNTL goes further by eliminating all structural tokens and putting status first for maximum signal survival.

7 Toward LNTL: An LLM-Native Tool Language

7.1 The Problem With How We Talk to Models

Right now, there are two ways to encode information for an LLM:

1. **JSON / structured data**—optimized for machines. Compact, parseable, but semantically flat.

false carries the same token weight as **true**. The signal is there but it’s quiet.

2. **Natural language**—optimized for humans. Rich, expressive, but verbose. “Unfortunately, the email sending operation encountered an error when attempting to connect to the SMTP server, which returned a 404 status code indicating the endpoint was not found.” That’s 35 tokens to say what could be said in 3.

Neither format is optimized for the actual consumer: a transformer that processes tokens through attention, where each token competes with every other token for influence on the output.

This raises a question: **what if we designed a language specifically for LLM-to-LLM and runtime-to-LLM communication?**

7.2 Design Principles

The goal is maximum semantic density per token—every token should carry as much meaning as possible, using the tokens that the model attends to most strongly.

Principle 1: High-salience tokens only

Not all tokens are equal in the attention mechanism. Uppercase words, rare words, and semantically loaded words get more attention. Design the language around them.

```
JSON (12 tokens, low salience):
{"success": false,
 "error": "API returned 404 Not Found"}

Natural language (25 tokens, diluted):
The email sending operation failed because
the API returned a 404 error.

LLM-native (6 tokens, high salience):
EMAIL FAIL 404 user@co.com NOSEND
```

Same information. 6 tokens instead of 12 or 25. Every token is a high-attention word.

Principle 2: Status-first ordering

Humans write narratives—context first, outcome last. “I tried to send the email to user@company.com but the server returned 404.” The model has to read 15 tokens before it learns the status.

LLM-native puts outcome first:

```
FAIL send_email user@co.com 404
OK create_pdf report.pdf 16KB
FAIL upload workspace timeout
```

The model sees FAIL/OK as the very first token of each line. The commitment horizon problem from §2.1 now works *for* us—the model commits to a failure narrative from token 1.

Principle 3: No filler tokens

Natural language is full of tokens that carry zero information for the model: “the”, “was”, “an”, “to”, “of”, “that”. These exist for human grammar but are noise for signal propagation. Every filler token is a slot where attention could have gone to a meaningful token instead.

```
Natural:
"The email to user@company.com was not
sent due to an error"

LLM-native:
"EMAIL->user@co.com NOSEND ERROR:404"
```

11 tokens vs 5. The information content is identical. The signal density doubles.

Principle 4: Semantic compression through convention

In natural language, “the operation completed successfully and the output file was saved to the documents folder in the workspace” takes 18 tokens. But if we establish conventions:

```
Convention: OK always means success.
            Path after arrow is location.
Format:    OK tool->location [size]
Result:    OK pdf->workspace/docs 16KB
```

4 tokens. The model learns the convention from the system prompt (stated once) and then every tool result is ultra-compact. This is like how HTTP headers work—

200 OK instead of “The server has successfully processed your request and is returning the requested resource.”

7.3 A Sketch of the Language

7.4 Why This Should Work (Information Theory)

Shannon’s channel capacity theorem tells us the maximum rate at which information can be reliably transmitted over a noisy channel:

$$C = B \cdot \log_2(1 + S/N) \quad (6)$$

where B is bandwidth (context window), S is signal power, and N is noise power.

For an LLM “channel”:

- B (bandwidth) = context window size in tokens. Fixed by the model (e.g., 200K tokens).
- S (signal) = semantic content of meaningful tokens. Varies by encoding.
- N (noise) = filler tokens, structural overhead, competing signals.

JSON encoding: lots of structural noise ($\{$, $\}$, $:$, $,$, commas) that consume tokens but carry no semantic content. S/N is low.

Natural language: grammatical filler (“the”, “was”, “an”) that consume attention but carry no task-relevant meaning. S/N is medium.

LLM-native: every token carries semantic content. Structural tokens eliminated. S/N is high.

By improving S/N , we increase the effective capacity of the context window without making it physically larger. A 4K context window with LLM-native encoding might carry as much *useful* information as an 8K window with JSON encoding.

7.5 The Compression Ratio

Back-of-envelope comparison for a typical tool result:

Format	Tokens	Signal	Density
JSON	12	3	25%
Natural language	25	4	16%
LLM-native	5	5	100%

Table 6: Signal density comparison across encoding formats.

In a session with 7 tool results:

- JSON: 84 tokens, 21 signal tokens
- Natural: 175 tokens, 28 signal tokens

```

# Status markers (always first token)
OK      -- action completed successfully
FAIL    -- action failed
PARTIAL -- partially completed
WAIT    -- action pending / needs approval
SKIP    -- action was not attempted

# Tool results
OK      send_email->user@co.com  [queued]
FAIL    send_email->user@co.com  [404:endpoint_missing]
OK      create_pdf->report.pdf   [16KB workspace:97]
FAIL    upload->workspace        [timeout 30s]
PARTIAL send_email->3/5sent      [2:bounced]

# Context passing between tasks
PRIOR task#371 OK create_excel->rankings.xlsx [workspace:97]
PRIOR task#372 FAIL send_email->prashanth@knobly.com [404]

# Verification pass (ledger summary)
LEDGER:
  OK      web_search "ICC rankings" [5 results]
  OK      create_excel->rankings.xlsx [workspace:97 doc_url:matsyaai.com/documents.php?id=97]
  FAIL    send_email->prashanth@knobly.com [404:endpoint_missing]
RESPOND: match LEDGER. FAIL=say failed. OK=say done.

```

Figure 3: Sketch of an LLM-Native Tool Language (LNTL). Status-first, high-salience, zero filler.

- LLM-native: 35 tokens, 35 signal tokens

That’s a **5x reduction** in context usage for the same information. In a long agentic session approaching context limits, this is the difference between the model remembering what happened in step 2 and forgetting it.

7.6 Open Questions

This is early thinking. Several things we don’t know yet:

1. **Does it actually improve model behavior?** The theory says yes—higher S/N , shorter chains, stronger signals. But we need empirical measurement. Run the same tasks with JSON results vs LLM-native results and compare hallucination rates.
2. **How much convention can the model hold?** If the system prompt defines 20 conventions (“OK means success, $\rightarrow$ means output location, $\square$ means metadata”), does the model reliably apply them all? Or does convention-recall have its own attention dilution problem?
3. **Does it hurt natural-sounding responses?** If the model is reading “FAIL send_email $\rightarrow$ user 404” as input, does it still produce a natural “I wasn’t able to send the email” as output? Or does the compressed input bleed into compressed output?
4. **Cross-model portability.** Does an LLM-native format that works well for Claude also work for GPT-4 or Gemini? Token vocabularies differ between

models, so a high-salience token in one model might not be in another.

5. Can the model learn to *produce* this format?

If tool results come in LLM-native, could the model also output in LLM-native when passing information to the next step in a multi-agent pipeline? That would give you compressed communication between agents, not just between runtime and agent.

This could become a real standard. Right now every agent framework (LangChain, CrewAI, AutoGPT, etc.) invents its own way to pass tool results to the model—JSON here, XML there, plain text somewhere else. It’s like how every database used to have its own query language until SQL came along and gave everyone one way to talk to any database. We think there’s room for the same thing here—a standard format for “tool talking to LLM” that every framework could adopt, so tools built for one agent work in any agent.

Existing tool protocols (like Anthropic’s MCP) standardize the *transport*—how tools connect, get called, and pass results back. But they don’t prescribe how the result content is formatted. The tool can still return `{"success": false}` and the model can still ignore it. Transport solves interop. Encoding solves reliability. They’re different layers.

8 Conclusion

LLM agents lie about what they did. Not because they don't have the information—but because five forces (word commitment, attention loss, training bias, quiet errors, and “get it done” instructions) all push toward claiming success.

The Action Ledger fixes this with a simple idea: keep a runtime-managed record of what actually happened, and when something failed, make the model look at that record and rewrite its answer.

The key insight: **the truth must come from outside the model**. The ledger is written by the runtime, not by the LLM. The verification pass uses the model's own ability—but on the runtime's terms, with the evidence in the strongest possible position.

This is:

- **Cheap**—zero cost when everything works, one API call when something fails
- **Effective**—eliminated hallucinated success in our production deployment
- **Simple**—about 30 lines of code
- **General**—works for any agent, any tools, any model

We think this will become standard. The same way assertions and contracts became standard in software engineering after Meyer, ground truth verification will become standard in agentic AI. Your agent should never lie to you about what it did.

9 Availability

The Action Ledger is implemented in BRU, an autonomous AI agent for business professionals. Code covers both cloud and local execution. Production deployment at matsyaai.com.

Prashanth Hebbur — prashanth@knobly.com
BRU — autonomous AI agent for business — matsyaai.com

References

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *Anthropic*, 2022.
- [2] Anirban Basu. Tool receipts, not zero-knowledge proofs: Practical hallucination detection for AI agents. *arXiv preprint arXiv:2603.10060*, 2026.
- [3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2023.
- [4] Elio Masciari et al. TOON: Token-oriented object notation. *arXiv preprint arXiv:2601.12014*, 2026.
- [5] Bertrand Meyer. Applying “design by contract”. *IEEE Computer*, 25(10):40–51, 1992.
- [6] Ethan Perez, Sam Ringer, Kamilė Lukošiuūtė, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. Discovering language model behaviors with model-written evaluations. *Anthropic*, 2022.
- [7] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2023.
- [8] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [10] Zhiheng Yuan et al. AEGIS: No tool call left unchecked. *arXiv preprint arXiv:2603.12621*, 2026.